

LABYRINTH — working implementation

A complete, runnable version of the Labyrinth game from your four documents.

The engine is **deterministic Python** (exactly as `04_PSEUDOCODE.md` specifies: *"Game Engine: deterministic, in-memory. Players: AI API calls."*). Every one of the 8 seats is independently a **Human, Groq, Gemini, Anthropic, OpenAI**, or offline **Heuristic** player — any mix, one line of configuration.

1. Quick start (no API key, no installs)

```
cd labyrinth_project
python3 run_game.py
```

That plays a full 8-player game with offline bots and prints the turn-by-turn transcript plus the final debrief. Nothing to install — pure standard library.

Check everything works:

```
python3 -m unittest discover -s tests -v      # 77 tests
```

2. Choosing who plays each seat

Open `run_game.py` and edit the `PLAYERS` list near the top. That is the only thing you need to change:

```
PLAYERS = [
    "human",
    "groq",
    "gemini",
    "human",
    "heuristic",
    "anthropic",
    "groq",
    "openai",
]
```

Each entry is Player 1 through Player 8, in order. Valid values: "human" , "heuristic" , "anthropic" , "gemini" , "groq" , "openai" .

```
PLAYERS = ["human"] * 8      # all human
PLAYERS = ["groq"] * 8      # all Groq
```

Save the file, run `python3 run_game.py` , and that exact combination plays — nothing else to touch.

Human seats need nothing installed; they just read the terminal (see §3). For AI seats, install and export a key for whichever providers you use:

```
pip install anthropic google-generativeai groq openai
export ANTHROPIC_API_KEY=sk-ant-...
export GOOGLE_API_KEY=...      # or GEMINI_API_KEY
export GROQ_API_KEY=...
export OPENAI_API_KEY=...
```

Every AI seat gets the exact same `rules/03_PLAYER_PROMPT.md` as its system prompt, the exact same JSON action schema, and the exact same perception payload each turn — only the network call underneath differs. If an API call fails mid-game, that seat falls back to the offline bot for that turn so the game never crashes mid-run.

3. Playing a human seat

A human seat prints the perception in plain, readable text and asks for one line back:

```
=====
PLAYER 4
=====
Location:
Workshop
Health:
Healthy
Inventory:
Knife
Visible Players:
Player 2
Player 5
Visible Items:
Magnifying Glass
Connected Rooms:
Storage Room
```

```
Laboratory
Objective:
Recover Drive B
=====
Enter your action (or 'help' for command examples):
>
```

Answer either with JSON —

```
{"action": "MOVE", "target": "Storage Room"}
```

— or with a simple command:

```
move Storage Room
search
attack 2
say I found nothing in here
```

Type `help` at the prompt any time to see the full list of simple commands. Plain text that isn't a recognised command or JSON is just spoken aloud (`{"action": "WAIT", "speech": "..."}`), so you can chat without memorising syntax. Either way, the result goes through the same unmodified `parse_action()` every other brain's output goes through.

4. All the run-time options

```
python3 run_game.py --help

--seed 42                reproducible game (same seed = same game)
--turns 60               turn limit safety valve
--key-model permanent|consumed  Vault Key behaviour (Engine Prompt 4.11)
--leave-after 32        turn after which heuristic bots give up and run
--quiet                hide the live transcript
--out results/          write transcript.txt and results.json
```

(Who controls each seat is not a flag — it's the `PLAYERS` list in the file, per §2.)

5. What each file does

```
labyrinth_project/
└─ run_game.py          PLAYERS list + command-line entry point
```

└─ labyrinth/	
└─ data.py	Appendix A rooms + map, Appendix E traits, roles, objectives, 5.17 durations, item lists
└─ models.py	canonical state: Room, Item, Player, GameState (9.
└─ actions.py	the action schema players speak, plus parsing
└─ perception.py	information filtering – the ONLY channel to a play
└─ engine.py	setup, turn loop, every mechanic, scoring (never imports labyrinth.brains – see §9)
└─ brains/	every kind of player, one shared contract
└─ base.py	BaseBrain: decide(perception) -> action
└─ human.py	HumanBrain: readable print + simple commands
└─ heuristic.py	HeuristicBrain: offline bot, no API key
└─ llm.py	LLMBrain: shared prompt/history/fallback logic
└─ anthropic_brain.py	AnthropicBrain(LLMBrain) – one method: _call_api
└─ gemini_brain.py	GeminiBrain(LLMBrain) – one method: _call_api
└─ groq_brain.py	GroqBrain(LLMBrain) – one method: _call_api
└─ openai_brain.py	OpenAIBrain(LLMBrain) – one method: _call_api
└─ __init__.py	PLAYER_TYPES: name -> class
└─ rules/	your source documents
└─ tests/	
└─ test_engine.py	43 mechanic-by-mechanic tests
└─ test_providers.py	34 tests for the brain/provider abstraction

6. How one turn runs (Master Rulebook 1.6)

1. Every living player is asked for one action **in parallel** — nobody sees anyone else's choice first (1.5).
2. Resolution, always in this order: speech → movement → room interactions → searching → item interactions → combat → environment → deaths → evidence → sound → traits → timed actions advance → escapes.
3. Objectives and exposure are checked.
4. Each player is told **only what their character perceived**.

7. Rule → code map

Rule	Where
1.6 turn order	engine.run_turn
1.7 scoring	engine.final_results

2.11 hiding	<code>engine.resolve_movement , engine.leave_hiding</code>
4.19 carry limit (4, or 6 for Scavenger)	<code>Player.inventory_limit</code>
5.12 / 5.17 interruptible actions	<code>engine.advance_interruptible_actions</code>
6.4 knife	<code>engine.knife_attack</code>
6.9 unarmed fight	<code>engine.begin_unarmed_fight , finish_unarmed_fight</code>
6.9 Final Record	<code>engine.write_final_record , complete_timed_action</code>
6.17 Observatory fall	<code>engine.push_fall</code>
7.x roles	<code>engine.claim_role</code> + bonuses in search/sound/traits
8.x objectives	<code>check_objective_completion , finalize_objective_on_escape</code>
9.4 / 9.5 information discipline	<code>perception.build_perception_payload</code>
9.6 simultaneous conflicts	contested pickups in <code>resolve_item_interactions</code>
Appendix C sound	<code>propagate_sound , rooms_within_hearing</code>
Appendix D drives/vault/record	<code>_mech_process_drive , _mech_destroy_drive , _mech_open_gate</code>
Appendix E traits	<code>activate_traits , trait_tokens</code>

8. Judgement calls I had to make

The rulebook leaves these to the Engine's discretion, so I picked one and made it consistent. All are one-line changes if you want them different.

1. **One turn = 15 in-world seconds** (`data.TURN_SECONDS`). So processing a drive (20s) takes 2 turns, destroying one (15s) takes 1, writing the Final Record (30s) takes 2, an unarmed fight (30s) takes 2.
2. **Vault Key = permanent** by default: the first person to use it opens the gate for everyone. `--key-model consumed` gives the alternative.

3. **Search success** is 60%, +25% for Investigator or Scavenger, –20% in the dark. Hidden players are found on the same roll.
4. **Knife**: kills outright against an unaware target; against an aware one it lands 70% of the time, otherwise the attacker is left Injured.
5. **Awareness (6.10)**: you are aware if you were acting against the attacker, fleeing or waiting; otherwise 50% (+30% for the Observer).
6. **Observatory edge** lasts the turn you lean out *and* the following turn — otherwise nobody could ever time a push.
7. **Traits** fire from the real internal state behind the declared action, and watchers notice them 60% of the time (100% for the Observer). The Actor suppresses their own tells half the time.
8. **Exposure** only counts accusations spoken aloud with someone else present.
9. **Turn limit** is a safety valve: anyone still inside scores 0, not –1.
10. **Objective 6** counts only *verified* role knowledge — you watched someone claim a role item, or they truthfully named their role. Being lied to records a belief, not knowledge.

9. The provider abstraction

Every seat, human or AI, is just an object satisfying one contract

```
( labyrinth/brains/base.py ):
```

```
class BaseBrain:
    def decide(self, perception: dict) -> dict | str:
        ...
```

The engine only ever calls `player.brain.decide(payload)` — it has no idea whether that's a person typing at a terminal, an offline bot, or which AI company answered the call.

`LabyrinthEngine.__init__` and `run_turn` never import anything from `labyrinth.brains` (verified by `tests/test_providers.py::test_engine_never_imports_the_brains_package`).

Turning a `PLAYERS` entry into a brain is one small function in `run_game.py`, no factories or plugin machinery involved:

```
def create_brain(player_type, player_id, prompt, seed, leave_after):
    cls = PLAYER_TYPES[player_type]          # a plain dict: name -> class
    if player_type == "human":
        return cls(player_id)
    if player_type == "heuristic":
```

```

        return cls(player_id, seed=seed, leave_after_turn=leave_after)
    fallback = HeuristicBrain(player_id, seed=seed, leave_after_turn=leave_after)
    return cls(player_id, system_prompt=prompt, fallback=fallback)

```

Any object with `decide(perception) -> dict` works, so writing your own player is this short:

```

from labyrinth import LabyrinthEngine
from labyrinth.brains import HeuristicBrain

class MyBrain:
    def __init__(self, pid): self.id = pid
    def decide(self, perception):
        return {"action": "SEARCH", "reasoning": "look around"}

brains = [MyBrain(1)] + [HeuristicBrain(i) for i in range(2, 9)]
results = LabyrinthEngine(brains, seed=7).run()

```

Adding a brand-new LLM provider

Every LLM-backed brain shares one base class, `LLMBrain` (`labyrinth/brains/llm.py`), which owns the Player Prompt, conversation history, JSON parsing, and falling back to the offline bot on failure. A new provider is a subclass that implements exactly one method:

```

# labyrinth/brains/my_provider_brain.py
from .llm import LLMBrain

class MyProviderBrain(LLMBrain):
    default_model = "my-provider-model-v1"

    def _call_api(self, system: str, messages: list[dict]) -> str:
        # messages is [{"role": "user"|"assistant", "content": str}, ...]
        # translate to your SDK's shape, call it, return the reply as text
        response = my_sdk.chat(system=system, messages=messages, model=self.model)
        return response.text

```

Then register it in `labyrinth/brains/__init__.py`:

```

from .my_provider_brain import MyProviderBrain
PLAYER_TYPES["my_provider"] = MyProviderBrain

```

Add `"my_provider"` to the `PLAYERS` list in `run_game.py` and it plays immediately — no engine, model, or perception changes needed. `tests/test_providers.py` proves this with a

stub provider (no network) that does exactly this and nothing more.

The offline bots are deliberately simple — they chase their objective and run for the exit, so games are quiet (about one death per twenty games). AI players negotiate, lie and kill far more. Combat itself is fully covered by the test suite either way.